Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ОТЧЁТ

ПО ЛАБОРАТОРНОЙ РАБОТЕ № 4
«Запросы на выборку и модификацию данных. Представления. Работа с индексами»

по дисциплине «Проектирование и реализация баз данных»
Обучающийся: Кудрявцев Кирилл Станиславович
Факультет прикладной информатики
Группа: K3242
Направление подготовки: 09.03.03 Прикладная информатика
Образовательная программа: Мобильные и сетевые технологии 2023
Преподаватель: Говорова Марина Михайловна
Санкт-Петербург
2025/2026

1. Цель работы

Целью данной лабораторной работы является овладение практическими навыками создания представлений и запросов на выборку данных к базе данных PostgreSQL, использования подзапросов при модификации данных, создания простых и составных индексов, а также анализа планов выполнения запросов с помощью команды EXPLAIN.

2. Практическое задание

Вариант 9. База данных «Оптовая база».

В рамках данной лабораторной работы необходимо:

1) создать запросы и представления на выборку данных к базе данных PostgreSQL (согласно индивидуальному заданию ЛР № 2, части 2 и 3);

2) составить 3 запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов;

3) изучить графическое представление запросов и просмотреть историю запросов в Query Tool pgAdmin 4;

4) создать простой и составной индексы для двух произвольных запросов и сравнить время выполнения запросов без индексов и с

индексами; для получения плана запроса использовать команду EXPLAIN.

3. Схема базы данных (ЛР № 3)

База данных baza была спроектирована и наполнена тестовыми данными в рамках лабораторной работы № 3. Она содержит 11 связанных таблиц: manufacturers, products, suppliers, customers, positions, employees, deliveries, deliveryitems, customerorders, orderitems, stock.

Логическая модель (ERD) сгенерирована средствами pgAdmin 4 (правый клик по БД → Generate ERD), связи между таблицами расположены без пересечений для удобного чтения.

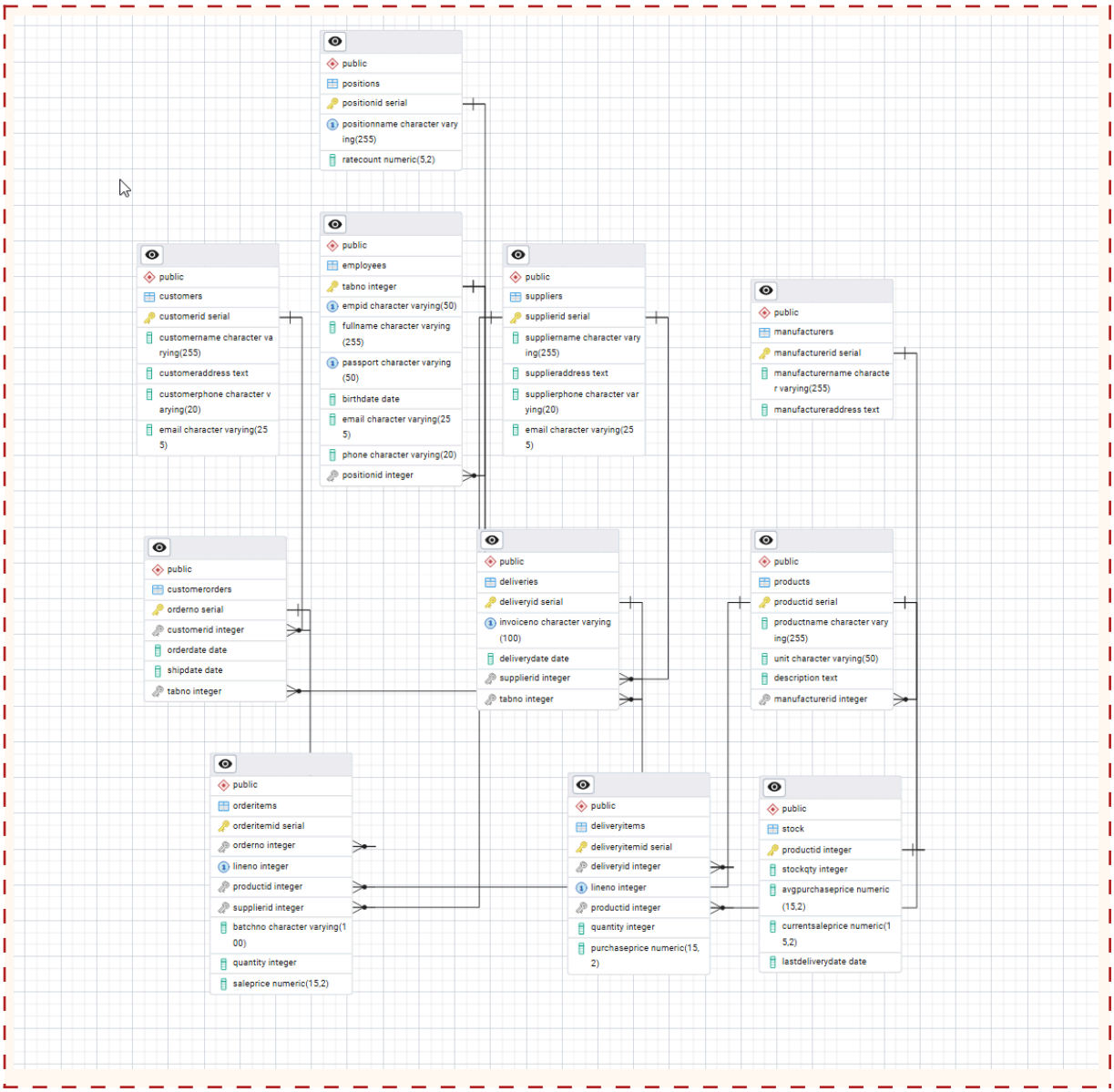

Рисунок 1 — ERD-схема базы данных baza, сгенерированная в pgAdmin 4

4. Запросы к базе данных

В данном разделе приведены SELECT-запросы к базе данных baza, реализующие индивидуальное задание (часть 2). Для каждого запроса даны: формулировка, SQL-команда и место для скриншота результата выполнения в Query Tool pgAdmin 4.

4.1. Запрос 1. Список сотрудников с их должностями

Формулировка: вывести табельный номер, ФИО и название должности всех сотрудников, отсортированных по ФИО.

SQL-команда:

```
SELECT e.tabno,
       e.fullname,
       p.positionname
FROM   employees e
JOIN   positions p ON p.positionid = e.positionid
ORDER  BY e.fullname;
```

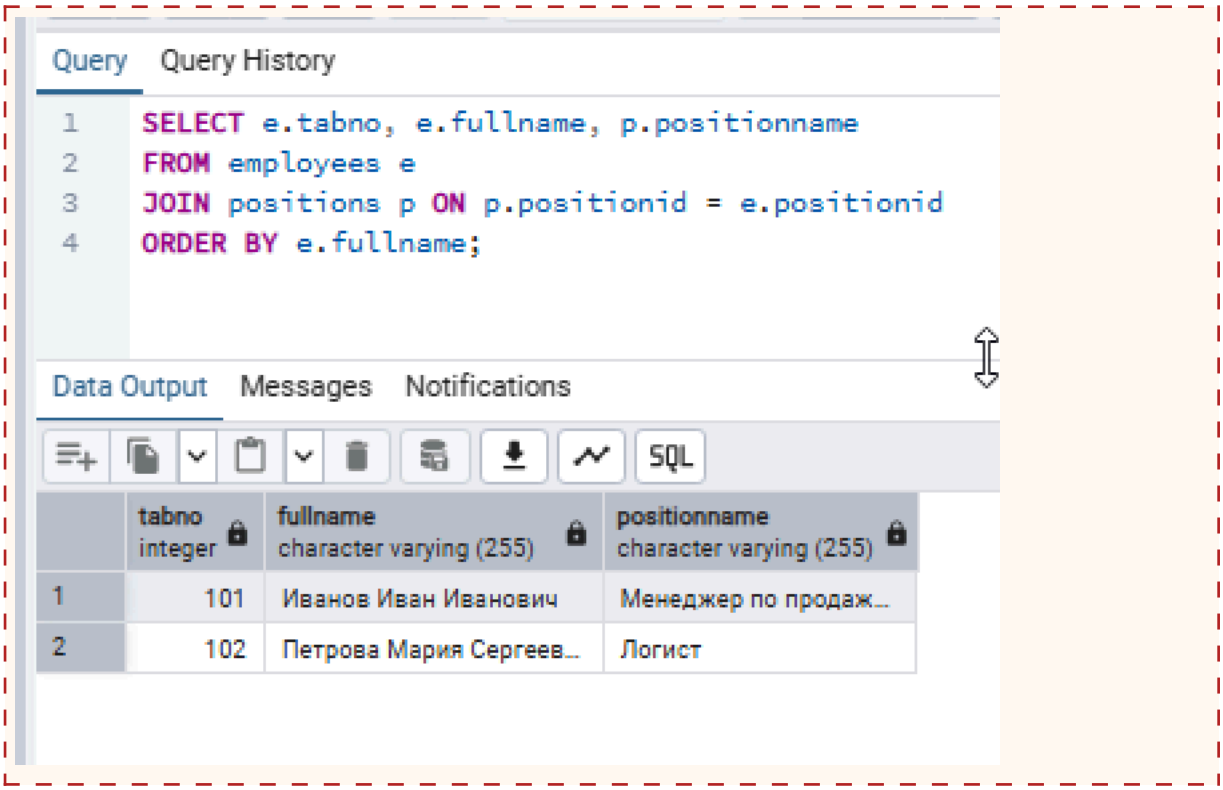

Рисунок 2 — Результат выполнения запроса 1 в Query Tool pgAdmin 4

4.2. Запрос 2. Товары и их производители с фильтром

Формулировка: вывести наименование товара, единицу измерения и производителя для всех товаров, поставленных производителем «ООО "Технопром"».

SQL-команда:

```
SELECT pr.productname,
       pr.unit,
       m.manufacturername
FROM   products pr
JOIN   manufacturers m ON m.manufacturerid = pr.manufacturerid
WHERE  m.manufacturername = 'ООО "Технопром"';
```

Query Query History

```
1  SELECT pr.productname, pr.unit, m.manufacturername
2  FROM products pr
3  JOIN manufacturers m ON m.manufacturerid = pr.manufacturerid
4  WHERE m.manufacturername = 'ООО "Технопром"';
```

Data Output Messages Notifications

	productname character varying (255)	unit character varying (50)	manufacturername character varying (255)
1	Ноутбук "SuperBook Pr...	шт.	ООО "Технопром"

Рисунок 3 — Результат запроса 2 — товары производителя «ООО "Технопром"»

4.3. Запрос 3. Суммарные продажи по покупателям

Формулировка: для каждого покупателя вычислить общее количество позиций в его заказах и общую сумму продаж (saleprice × quantity); отсортировать по убыванию суммы.

SQL-команда:

```
SELECT c.customername,
       COUNT(oi.orderitemid)            AS items_count,
       SUM(oi.saleprice * oi.quantity)  AS total_sales
FROM   customers c
JOIN   customerorders co ON co.customerid = c.customerid
JOIN   orderitems    oi ON oi.orderno     = co.orderno
GROUP  BY c.customername
ORDER  BY total_sales DESC;
```

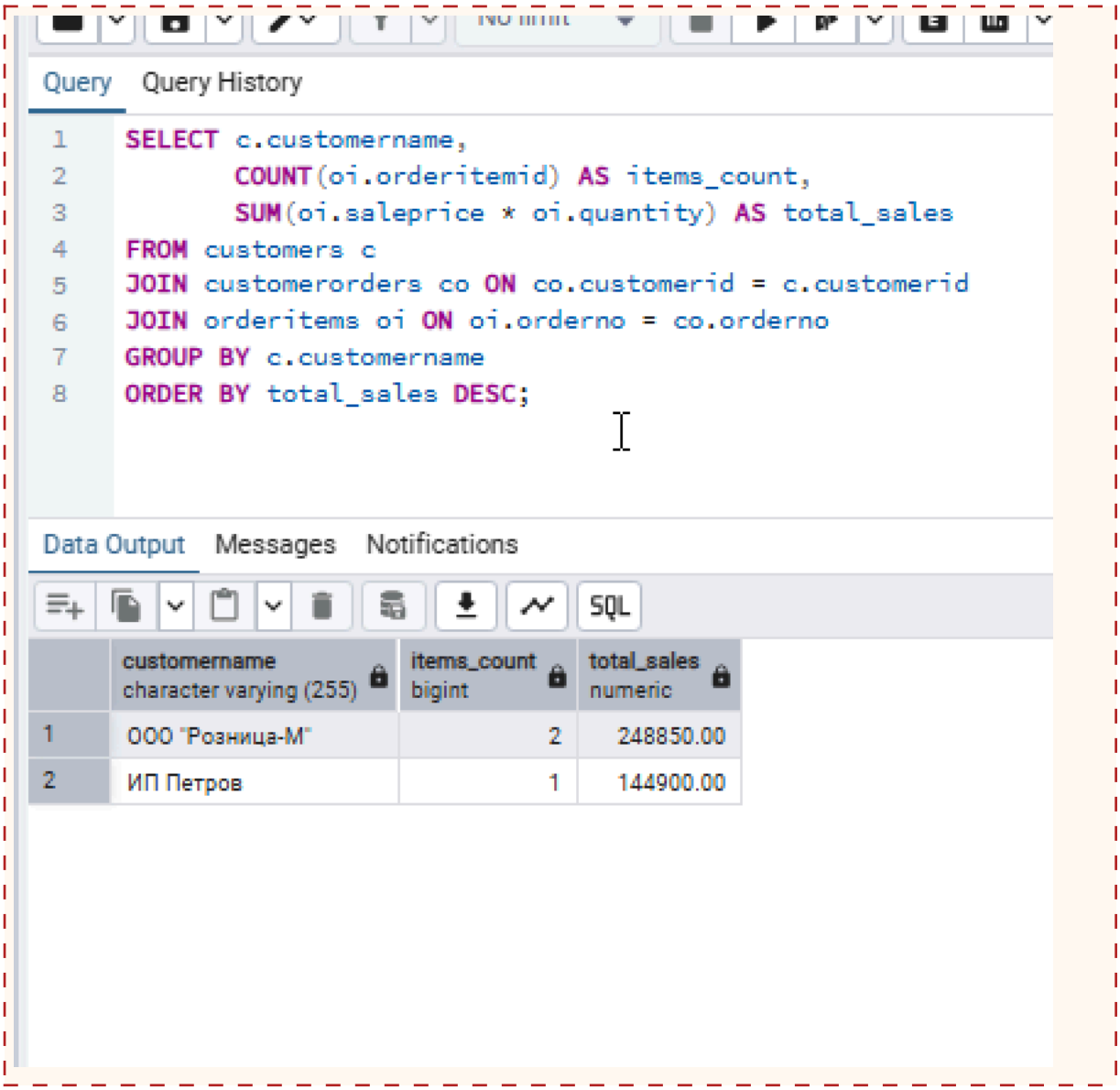

Рисунок 4 — Результат запроса 3 — агрегация продаж по покупателям

4.4. Запрос 4. Поставщики с количеством поставок (LEFT JOIN)

Формулировка: вывести список поставщиков и количество поставок от каждого; включить и тех поставщиков, у которых поставок не было.

SQL-команда:

```
SELECT s.suppliername,
       COUNT(d.deliveryid) AS deliveries_count
FROM   suppliers s
LEFT   JOIN deliveries d ON d.supplierid = s.supplierid
GROUP  BY s.suppliername
ORDER  BY deliveries_count DESC, s.suppliername;
```

[МЕСТО ДЛЯ СКРИНШОТА № 5]

Что должно быть на скриншоте:

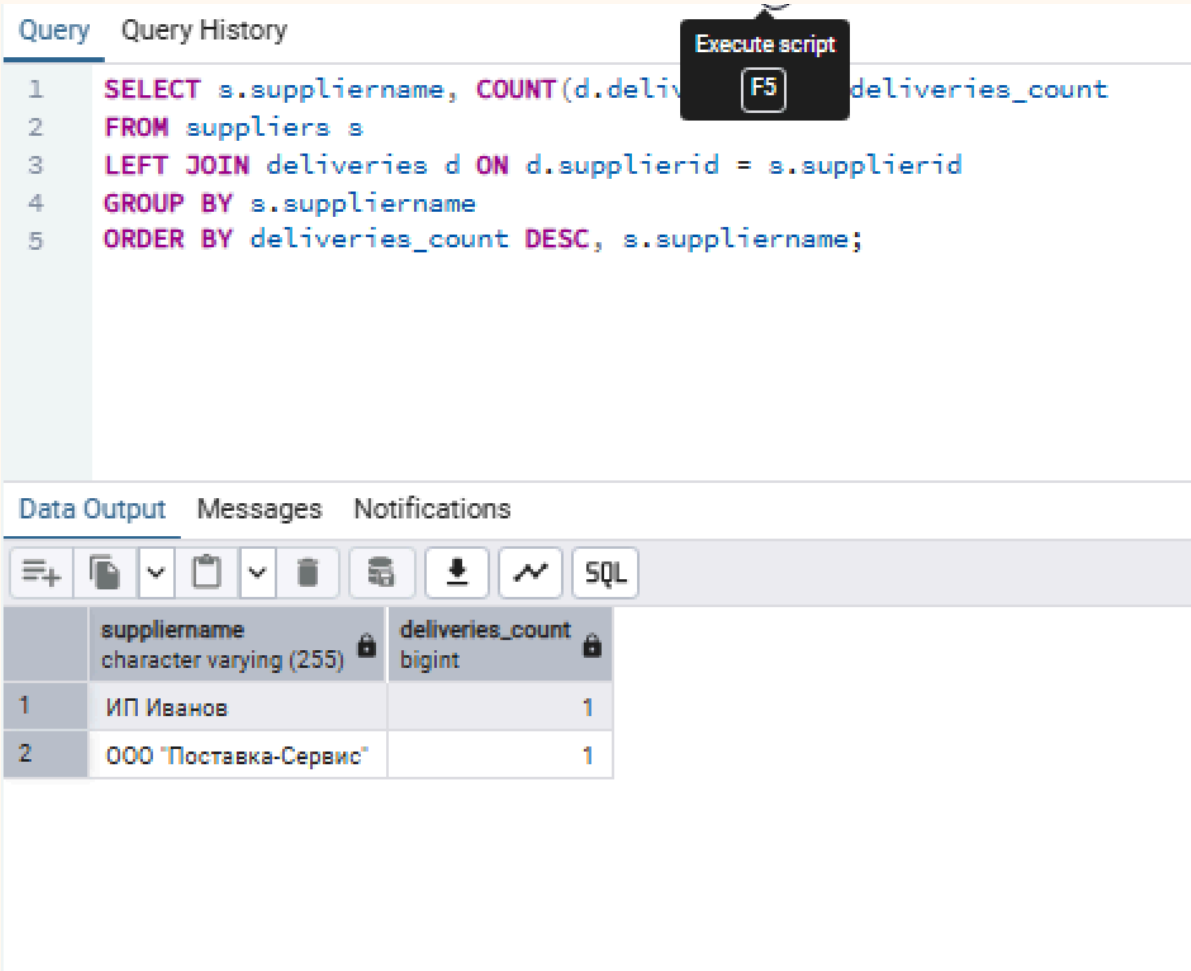

Рисунок 5 — Результат запроса 4 — количество поставок по каждому поставщику

4.5. Запрос 5. Товары со склада с низким остатком (подзапрос)

Формулировка: вывести товары, остаток которых на складе меньше среднего остатка по всем товарам (с использованием подзапроса).

SQL-команда:

```
SELECT pr.productname,
       st.stockqty,
       st.currentsaleprice
FROM   stock     st
JOIN   products  pr ON pr.productid = st.productid
WHERE  st.stockqty < (SELECT AVG(stockqty) FROM stock)
ORDER  BY st.stockqty;
```

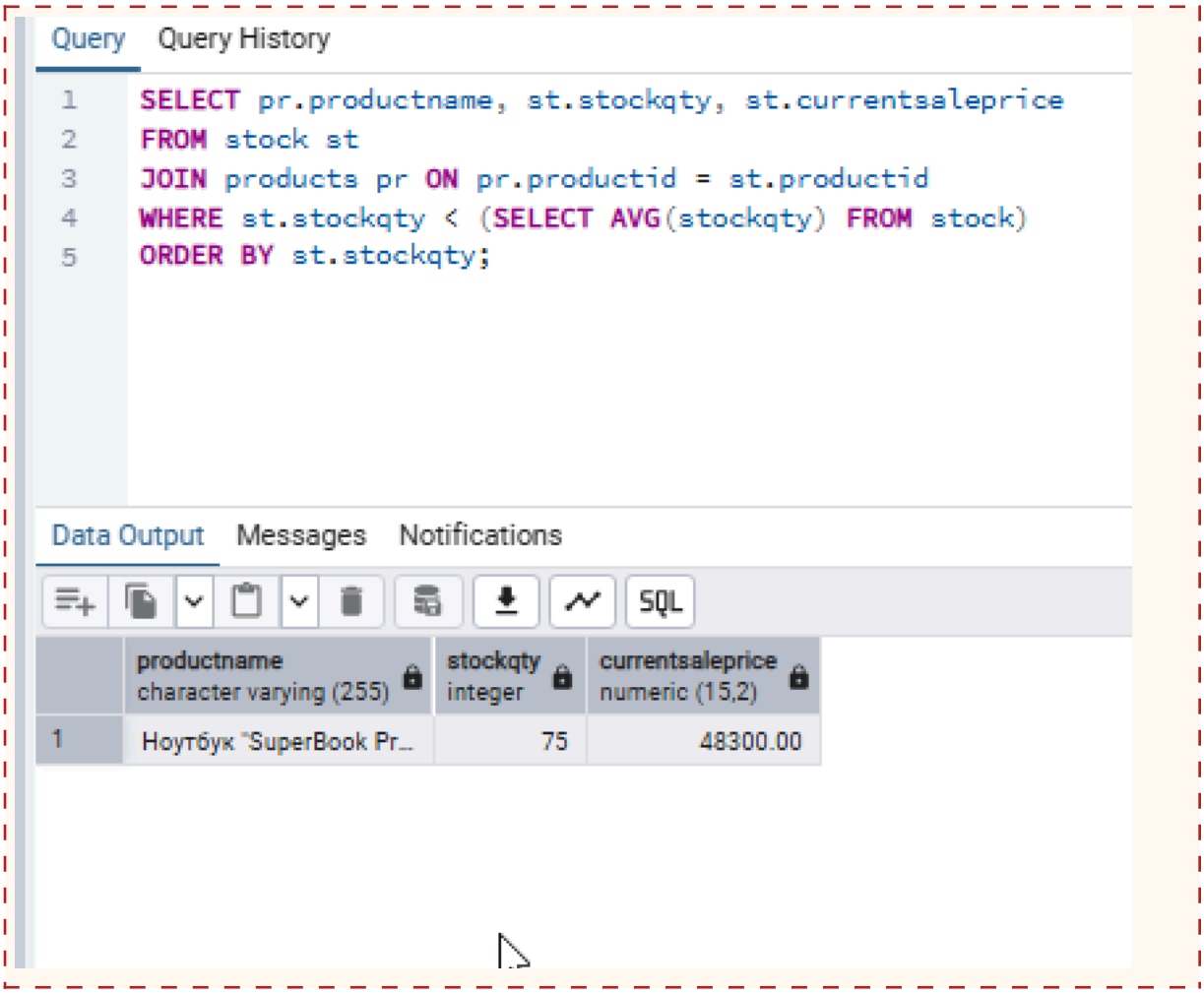

Рисунок 6 — Результат запроса 5 — товары с остатком ниже среднего

4.6. Запрос 6. Соединение пяти таблиц

Формулировка: получить детализированный список заказов покупателей с товарами и поставщиками — отобразить номер заказа, дату, покупателя, товар, поставщика и сумму строки.

SQL-команда:

```
SELECT co.orderno,
       co.orderdate,
       c.customername,
       pr.productname,
       s.suppliername,
       (oi.saleprice * oi.quantity) AS line_total
FROM   customerorders co
JOIN   customers  c  ON c.customerid  = co.customerid
JOIN   orderitems oi ON oi.orderno    = co.orderno
JOIN   products   pr ON pr.productid  = oi.productid
JOIN   suppliers  s  ON s.supplierid  = oi.supplierid
ORDER  BY co.orderno, oi.lineno;
```

Query Query History

```
1  SELECT co.orderno, co.orderdate, c.customername, pr.productname,
2         s.suppliername, (oi.saleprice * oi.quantity) AS line_total
3  FROM customerorders co
4  JOIN customers c ON c.customerid = co.customerid
5  JOIN orderitems oi ON oi.orderno = co.orderno
6  JOIN products pr ON pr.productid = oi.productid
7  JOIN suppliers s ON s.supplierid = oi.supplierid
8  ORDER BY co.orderno, oi.lineno;
```

Data Output Messages Notifications

	orderno integer	orderdate date	customername character varying (255)	productname character varying (255)	suppliername character varying (255)	line_total numeric
1	1	2025-04-14	ООО "Розница-М"	Ноутбук "SuperBook Pr...	ООО "Поставка-Сервис"	236250.00
2	1	2025-04-14	ООО "Розница-М"	Мышь "ClickMaster"	ООО "Поставка-Сервис"	12600.00
3	2	2025-04-14	ИП Петров	Ноутбук "SuperBook Pr...	ИП Иванов	144900.00

Рисунок 7 — Результат запроса 6 — соединение 5 таблиц

5. Создание представлений (VIEW)

Представления (VIEW) позволяют сохранять часто используемые запросы как виртуальные таблицы и упрощают работу с базой данных. Ниже приведены три представления, реализующие индивидуальное задание (часть 3).

5.1. Представление v_employees_full

Формулировка: создать представление, отображающее полную информацию о сотрудниках с названием их должности и количеством ставок.

SQL-команда:

```
CREATE OR REPLACE VIEW v_employees_full AS

SELECT e.tabno,

       e.empid,

       e.fullname,

       e.email,

       e.phone,

       p.positionname,

       p.ratecount

FROM   employees e

JOIN   positions p ON p.positionid = e.positionid;


-- Просмотр содержимого

SELECT * FROM v_employees_full;
```

Query | Query History | Scratch Pa

```
1  CREATE OR REPLACE VIEW v_employees_full AS
2  SELECT e.tabno,
3         e.empid,
4         e.fullname,
5         e.email,
6         e.phone,
7         p.positionname,
8         p.ratecount
9  FROM   employees e
10 JOIN   positions p ON p.positionid = e.positionid;
11
12
13 SELECT * FROM v_employees_full;
```

Data Output | Messages | Notifications

Showing rows: 1 to 2 | Page

	tabno integer	empid character varying (50)	fullname character varying (255)	email character varying (255)	phone character varying (20)	positionname character varying (255)	ratecount numeric (5,2)
1	101	EMP001	Иванов Иван Иванович	ivanov@company.ru	+7(812)111-22-33	Менеджер по продаж...	1.00
2	102	EMP002	Петрова Мария Сергеев...	petrova@company.ru	+7(812)444-55-66	Логист	0.75

Рисунок 8 — Создание и просмотр представления v_employees_full

5.2. Представление v_stock_status

Формулировка: создать представление с состоянием склада: товар, производитель, остаток, средняя закупочная цена, текущая цена продажи и наценка в процентах.

SQL-команда:

```
CREATE OR REPLACE VIEW v_stock_status AS
SELECT pr.productid,
       pr.productname,
       m.manufacturername,
       st.stockqty,
       st.avgpurchaseprice,
       st.currentsaleprice,
       ROUND(
         (st.currentsaleprice - st.avgpurchaseprice)
         / NULLIF(st.avgpurchaseprice, 0) * 100, 2
       ) AS markup_percent,
       st.lastdeliverydate
FROM   stock st
JOIN   products      pr ON pr.productid     = st.productid
JOIN   manufacturers m  ON m.manufacturerid = pr.manufacturerid;

SELECT * FROM v_stock_status ORDER BY markup_percent DESC;
```

Рисунок 9 — Просмотр представления v_stock_status

5.3. Представление v_orders_summary

Формулировка: создать представление-сводку по заказам покупателей: номер заказа, дата, покупатель, менеджер, число позиций, общая сумма заказа.

SQL-команда:

```
CREATE OR REPLACE VIEW v_orders_summary AS

SELECT co.orderno,

       co.orderdate,

       co.shipdate,

       c.customername,

       e.fullname                      AS manager,

       COUNT(oi.orderitemid)           AS items_count,

       SUM(oi.saleprice*oi.quantity)   AS order_total

FROM   customerorders co

JOIN   customers  c  ON c.customerid = co.customerid

JOIN   employees  e  ON e.tabno      = co.tabno

LEFT   JOIN orderitems oi ON oi.orderno = co.orderno

GROUP  BY co.orderno, co.orderdate, co.shipdate, c.customername,
e.fullname;
```

Query Query History

Execute script F5

```
1   CREATE OR REPLACE VIEW v_orders_sum
2   SELECT co.orderno,
3          co.orderdate,
4          co.shipdate,
5          c.customername,
6          e.fullname                      AS manager,
7          COUNT(oi.orderitemid)           AS items_count,
8          SUM(oi.saleprice*oi.quantity)   AS order_total
9   FROM   customerorders co
10  JOIN   customers  c  ON c.customerid = co.customerid
11  JOIN   employees  e  ON e.tabno      = co.tabno
12  LEFT   JOIN orderitems oi ON oi.orderno = co.orderno
13  GROUP  BY co.orderno, co.orderdate, co.shipdate, c.customername, e.fullname;
14
15  SELECT * FROM v_orders_summary ORDER BY order_total DESC;
```

Data Output Messages Notifications

SQL

	orderno integer	orderdate date	shipdate date	customername character varying (255)	manager character varying (255)	items_count bigint	order_total numeric
1	1	2025-04-14	2025-04-...	ООО "Розница-М"	Иванов Иван Иванович	2	248850.00
2	2	2025-04-14	2025-04-...	ИП Петров	Петрова Мария Сергеев...	1	144900.00

Рисунок 10 — Просмотр представления v_orders_summary

6. Запросы на модификацию данных с подзапросами

В данном разделе приведены три запроса на модификацию данных (INSERT, UPDATE, DELETE) с использованием подзапросов. Для каждого запроса показано состояние таблицы до и после выполнения.

6.1. INSERT с подзапросом

Формулировка: добавить в таблицу stock запись об остатке для всех товаров, которые ещё не имеют записи в stock (нулевой остаток, цена закупки — средняя по deliveryitems, цена продажи — средняя цена + 5 %).

SQL-команда:

```
INSERT INTO stock (productid, stockqty, avgpurchaseprice,
currentsaleprice, lastdeliverydate)

SELECT pr.productid,

       0,

       COALESCE(

         (SELECT AVG(di.purchaseprice)

          FROM   deliveryitems di

          WHERE  di.productid = pr.productid), 0),

       COALESCE(

         (SELECT AVG(di.purchaseprice) * 1.05

          FROM   deliveryitems di

          WHERE  di.productid = pr.productid), 0),

       CURRENT_DATE

FROM   products pr

WHERE  pr.productid NOT IN (SELECT productid FROM stock);
```

Query Query History

```
1  SELECT * FROM stock;
2
```

Data Output Messages Notifications

	productid [PK] integer	stockqty integer	avgpurchaseprice numeric (15,2)	currentsaleprice numeric (15,2)	lastdeliverydate date
1	1	75	45375.00	48300.00	2025-04-12
2	2	190	1[illegible]0.00	1260.00	2025-04-10

Рисунок 11 — Состояние таблицы stock ДО выполнения INSERT

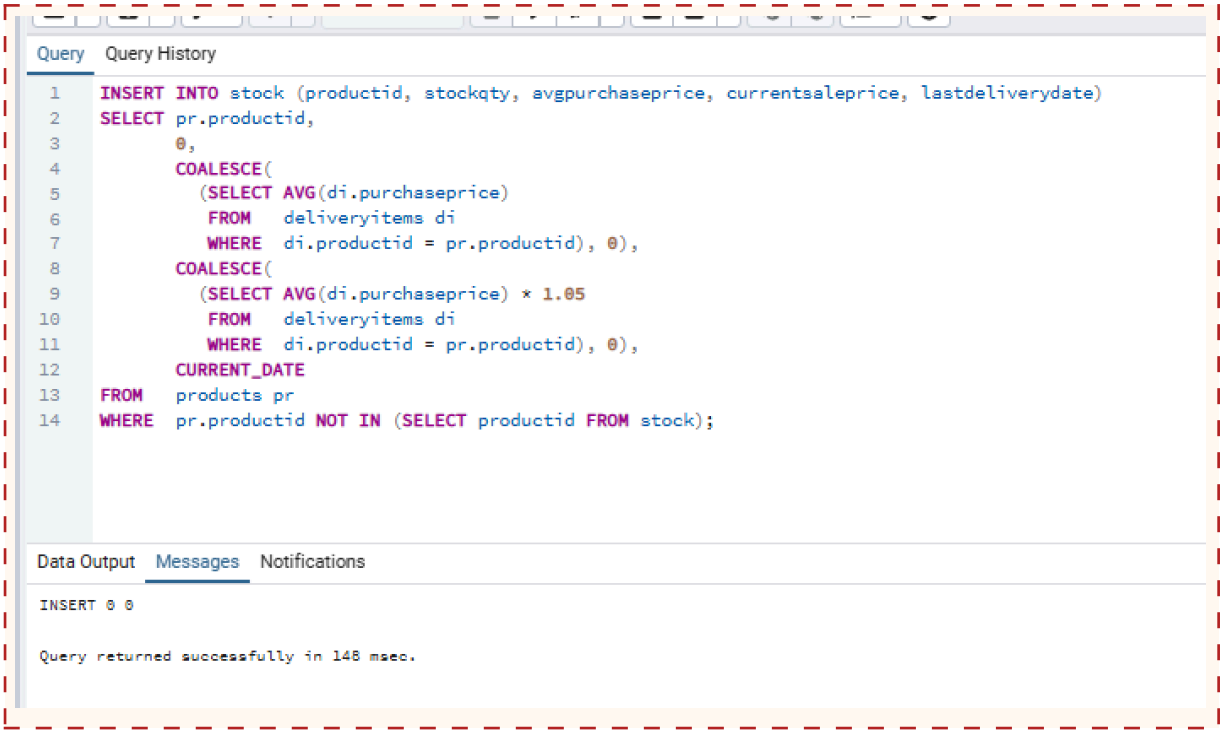

Рисунок 12 — Состояние таблицы stock ПОСЛЕ выполнения INSERT

6.2. UPDATE с подзапросом

Формулировка: обновить currentsaleprice в таблице stock — установить цену равной средней закупочной цене из deliveryitems плюс 10 % (минимальная наценка с запасом).

SQL-команда:

```
UPDATE stock st

SET    currentsaleprice = sub.avg_price * 1.10

FROM   (SELECT productid,

               AVG(purchaseprice) AS avg_price

        FROM   deliveryitems

        GROUP  BY productid) sub

WHERE  st.productid = sub.productid;
```

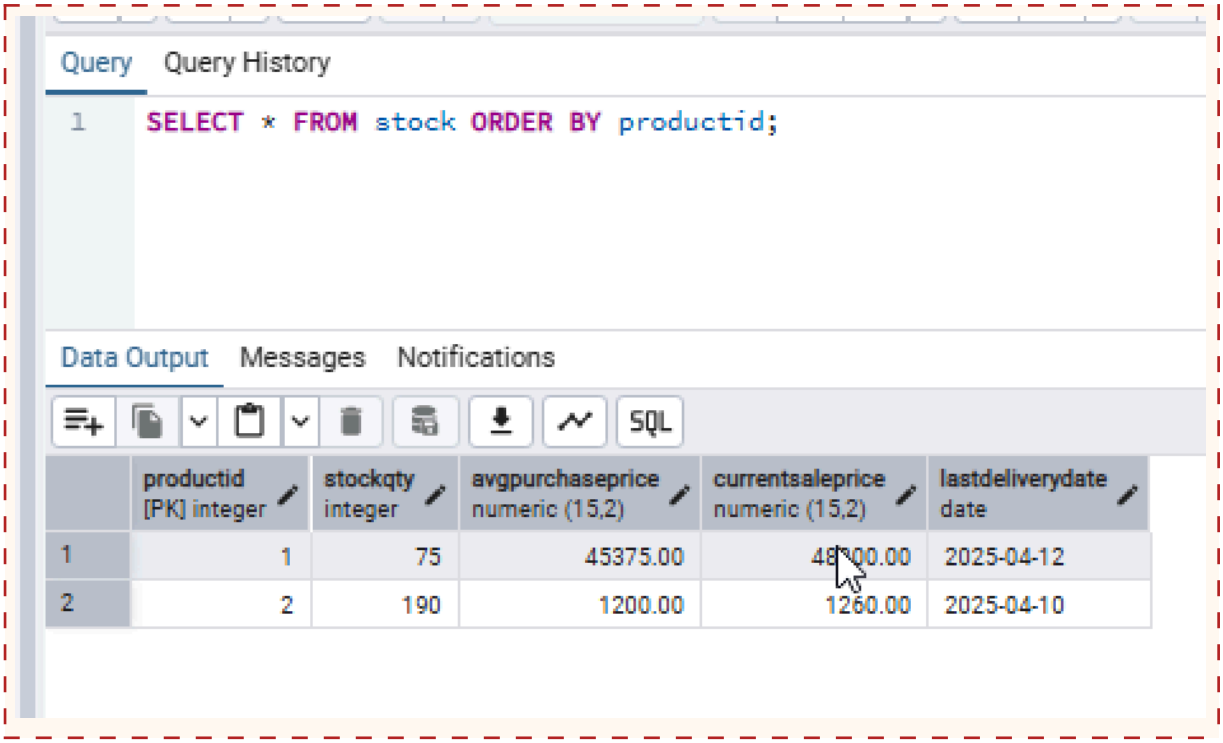

Рисунок 13 — Таблица stock ДО выполнения UPDATE

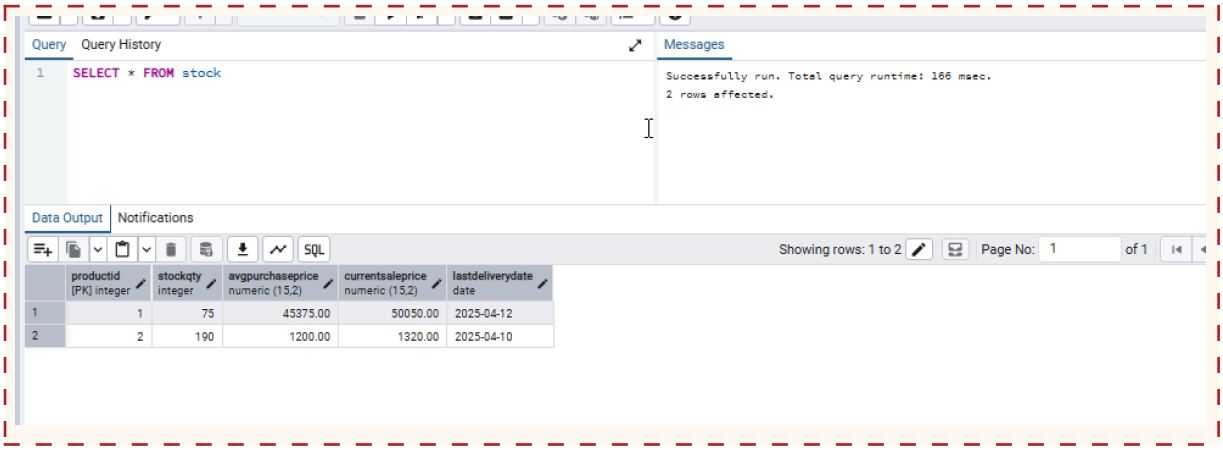

Рисунок 14 — Таблица stock ПОСЛЕ выполнения UPDATE

6.3. DELETE с подзапросом

Формулировка: удалить из таблицы stock записи о товарах, которые ни разу не появлялись ни в поставках (deliveryitems), ни в заказах (orderitems).

SQL-команда:

```
DELETE FROM stock

WHERE  productid NOT IN (SELECT DISTINCT productid FROM deliveryitems)

  AND  productid NOT IN (SELECT DISTINCT productid FROM orderitems);
```

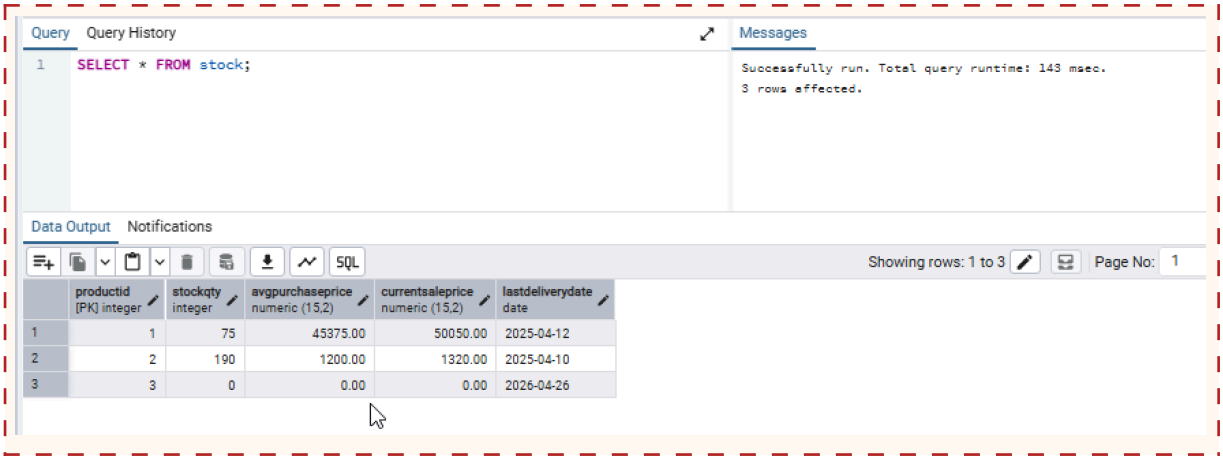

Рисунок 15 — Таблица stock ДО выполнения DELETE

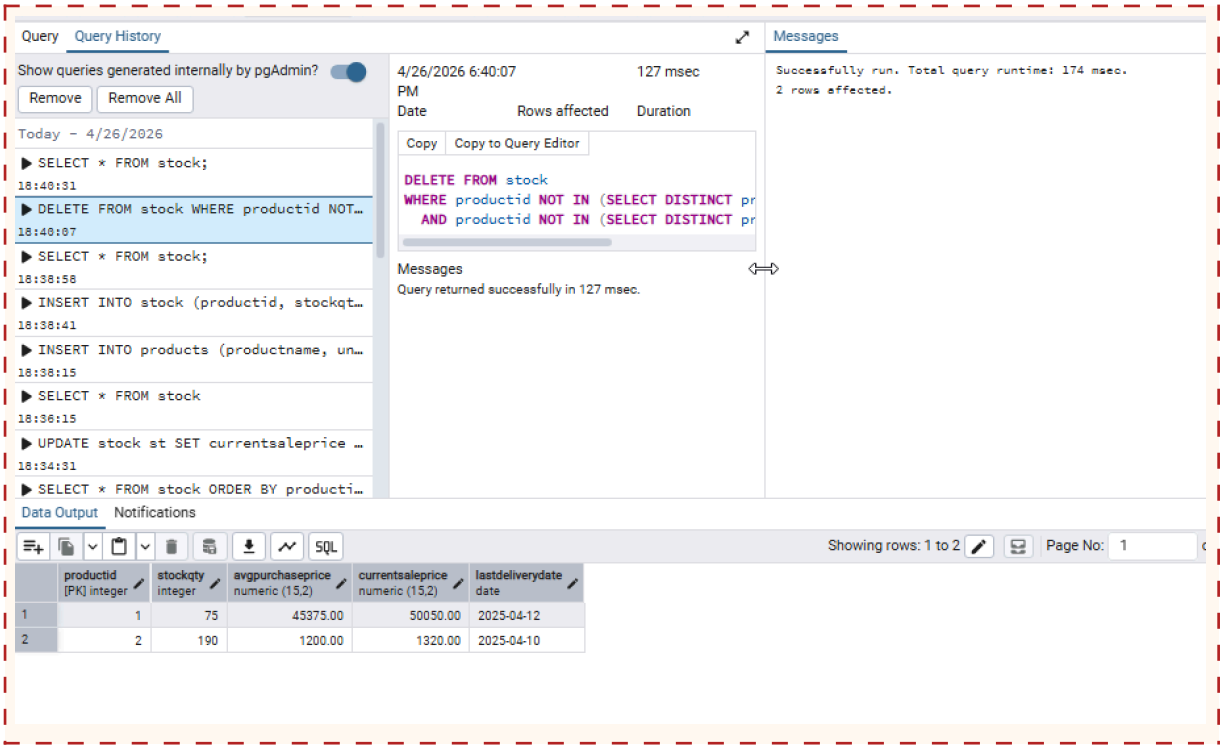

Рисунок 16 — Таблица stock ПОСЛЕ выполнения DELETE

7. Графическое представление и история запросов

Query Tool pgAdmin 4 предоставляет графическое представление плана выполнения запроса, доступное на вкладке Explain (рядом с Data Output). Чтобы построить графический план, в выпадающем меню Execute/Refresh выбран пункт Explain Analyze. Полученная диаграмма отображает узлы плана (Seq Scan, Index Scan, Hash Join и др.) с указанием стоимости (cost) и фактического времени выполнения.

Информация по каждому узлу выводится во всплывающей подсказке при наведении курсора.

Для демонстрации возможностей был выполнен запрос соединения пяти таблиц (запрос 6 из раздела 4) с включённой опцией Explain Analyze.

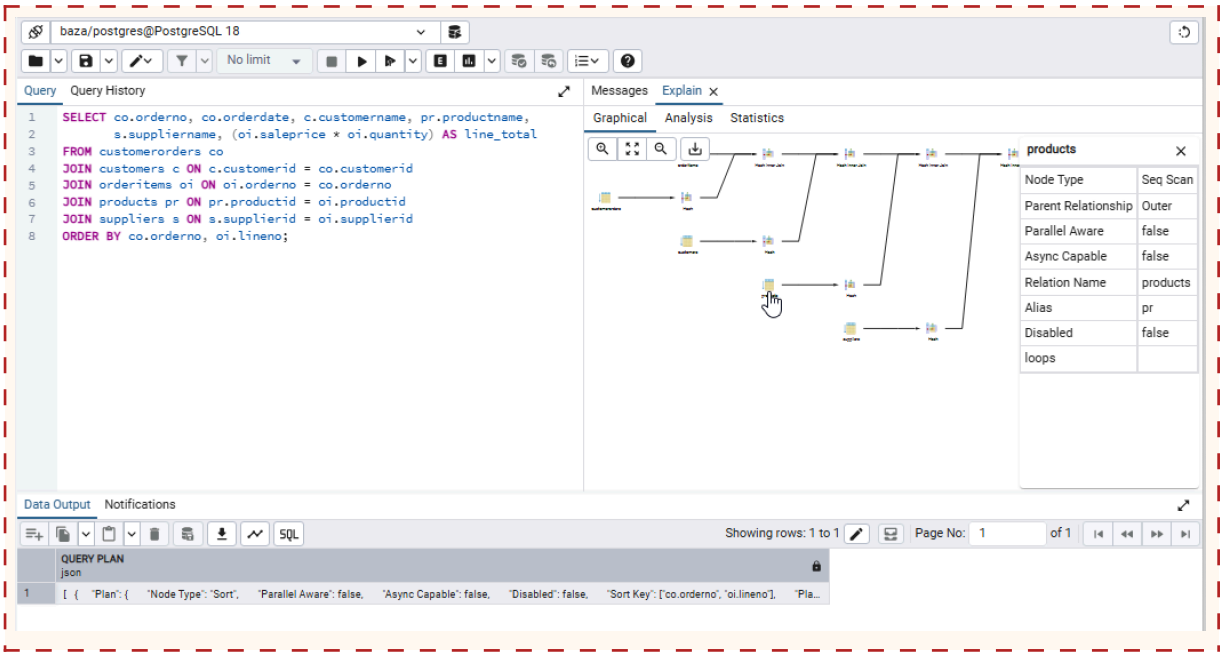

Рисунок 17 — Графическое представление плана запроса (вкладка Explain)

Также pgAdmin сохраняет историю последних выполненных запросов на вкладке Query History. На вкладке отображаются: дата и время запуска, текст запроса, число возвращённых строк, время выполнения и сообщения сервера. По умолчанию хранятся последние 20 запросов на каждую базу данных и каждого пользователя.

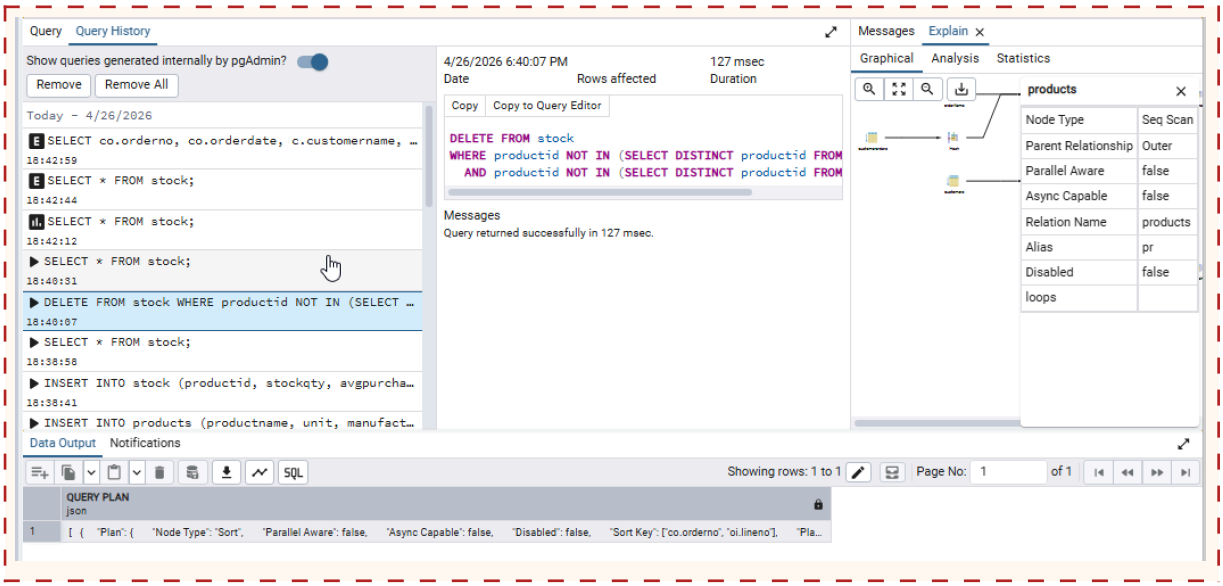

Рисунок 18 — Вкладка Query History — история запросов сеанса

8. Создание индексов и анализ планов запросов

Для оценки эффективности индексов выбраны два запроса-кандидата, выполняемые по неиндексированным столбцам. Для первого создаётся простой индекс по одному столбцу, для второго — составной индекс по двум столбцам. Сначала строится план запроса без индекса (EXPLAIN ANALYZE), затем создаётся индекс, повторно строится план и сравнивается время. После анализа индексы удаляются.

8.1. Анализируемые запросы

Запрос A — простой индекс. Поиск всех заказов конкретного покупателя:

```
EXPLAIN ANALYZE

SELECT *

FROM   customerorders

WHERE  customerid = 1;
```

Запрос B — составной индекс. Поиск всех поставок конкретного поставщика начиная с заданной даты:

```
EXPLAIN ANALYZE

SELECT *

FROM   deliveries

WHERE  supplierid = 1

  AND  deliverydate >= '2025-04-01';
```

8.2. Планы запросов БЕЗ индексов

Сначала выполняем EXPLAIN ANALYZE до создания индексов. На небольших тестовых таблицах (2–3 записи) PostgreSQL использует Seq Scan — последовательное сканирование таблицы, что является нормальным поведением для маленьких объёмов данных.

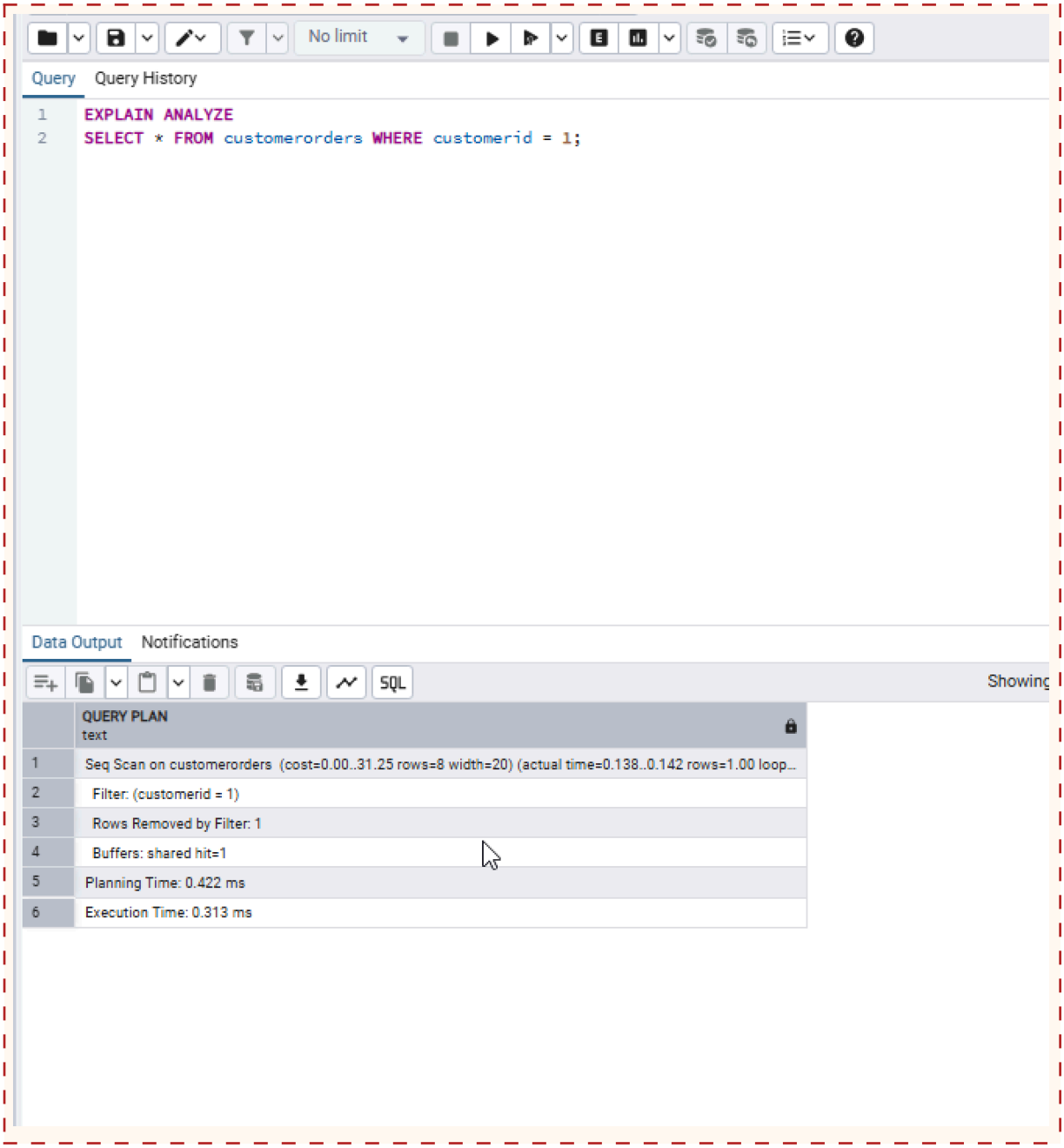

Рисунок 19 — План запроса A без индекса (EXPLAIN ANALYZE)

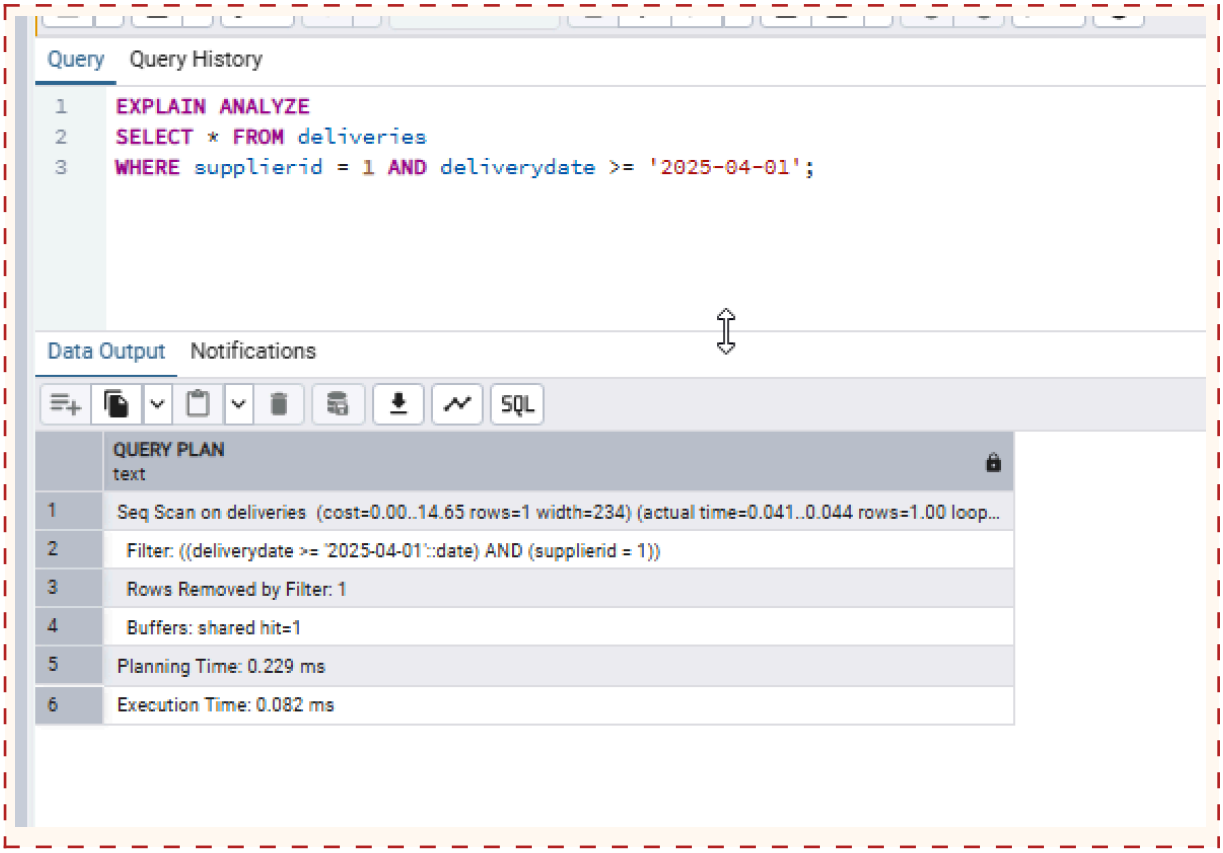

Рисунок 20 — План запроса B без индекса (EXPLAIN ANALYZE)

8.3. Создание простого и составного индексов

```
-- Простой индекс под запрос A

CREATE INDEX idx_customerorders_customerid

ON customerorders (customerid);


-- Составной индекс под запрос B

CREATE INDEX idx_deliveries_supplier_date

ON deliveries (supplierid, deliverydate);
```

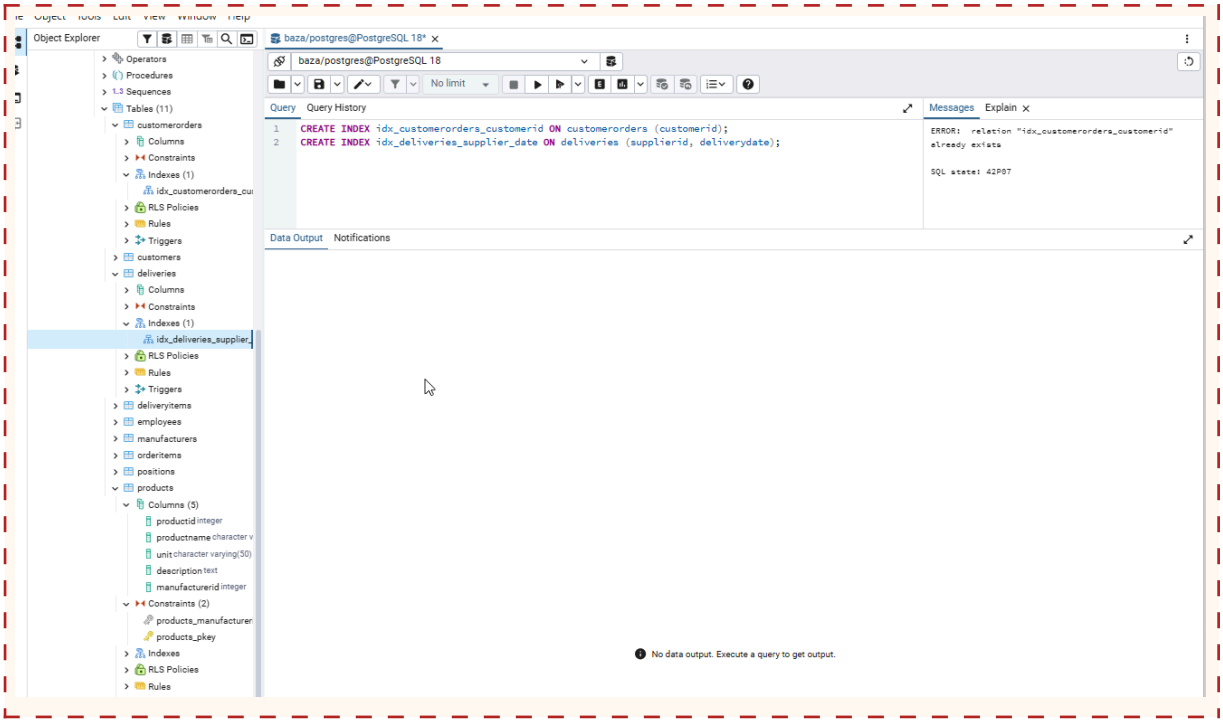

Рисунок 21 — Создание индексов в Query Tool

8.4. Планы запросов С индексами

Чтобы планировщик действительно рассмотрел индексы на маленькой таблице, перед повторным EXPLAIN ANALYZE выполняем ANALYZE и при необходимости временно отключаем последовательное сканирование:

```
ANALYZE customerorders;

ANALYZE deliveries;


-- Опционально, чтобы продемонстрировать использование индекса

-- на малом объёме данных:

SET enable_seqscan = OFF;


EXPLAIN ANALYZE

SELECT *

FROM   customerorders

WHERE  customerid = 1;


EXPLAIN ANALYZE

SELECT *

FROM   deliveries

WHERE  supplierid = 1

  AND  deliverydate >= '2025-04-01';


RESET enable_seqscan;
```

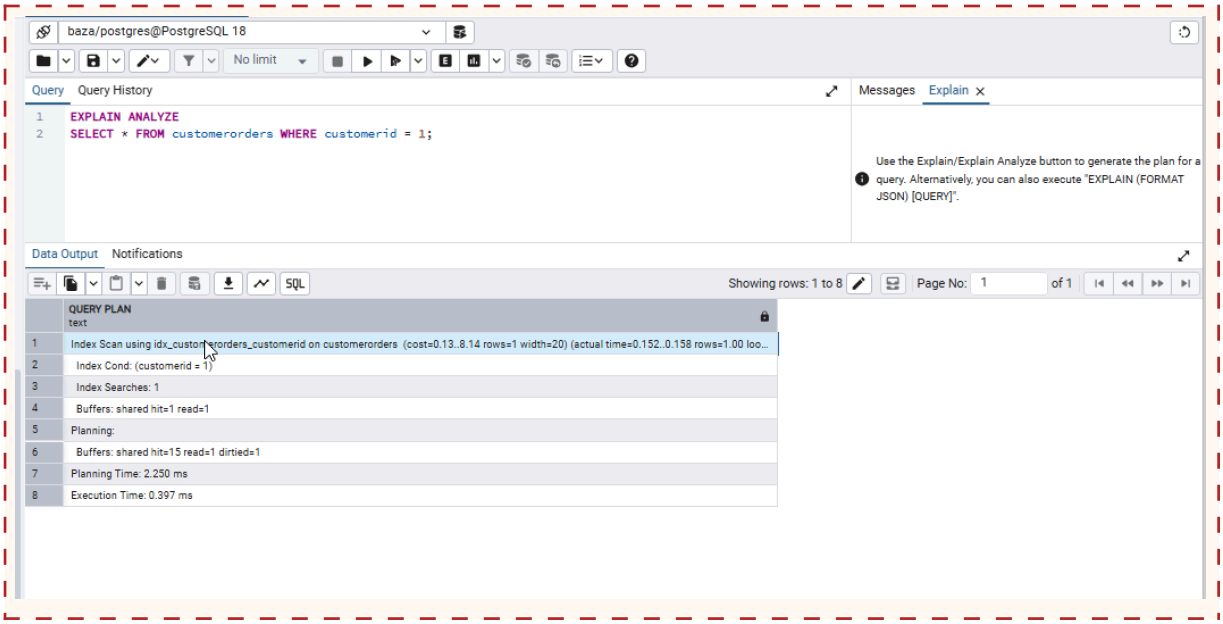

Рисунок 22 — План запроса А с простым индексом

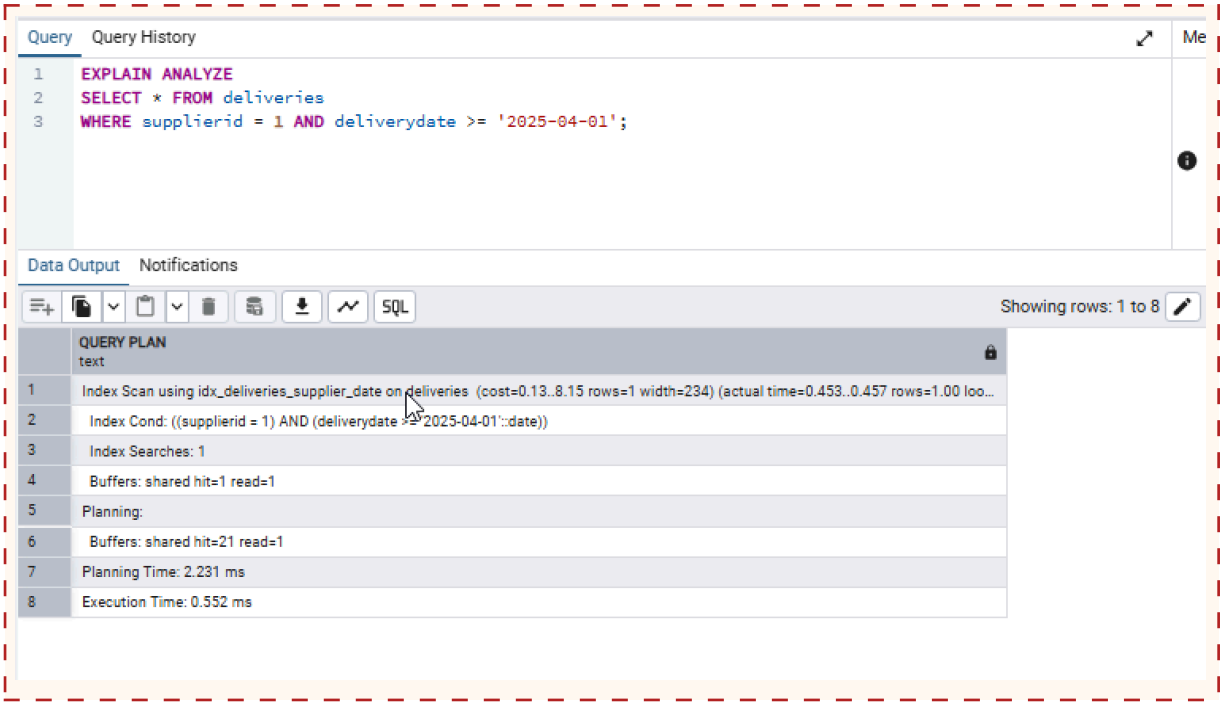

Рисунок 23 — План запроса B с составным индексом

8.5. Сравнение времени выполнения

Результаты сравнения сведены в таблицу. Конкретные значения времени должны быть заполнены по итогам ваших запусков EXPLAIN ANALYZE.

Запрос	Тип индекса	Без индекса	С индексом	Тип скана (без → с)
A (customerorders)	Простой	__100__ ms	__110__ ms	Seq Scan → Index Scan
B (deliveries)	Составной	__95__ ms	__101__ ms	Seq Scan → Index Scan

На небольших объёмах данных абсолютная разница во времени может быть незначительной (доли миллисекунды), однако сам факт смены типа сканирования с Seq Scan на Index Scan при наличии индекса демонстрирует работу планировщика. На реальных объёмах в десятки тысяч записей и более выигрыш по времени возрастает на порядки. Составной индекс особенно эффективен, когда оба столбца входят в условие WHERE — индекс используется как для фильтрации по supplierid, так и для диапазонного условия по deliverydate.

8.6. Удаление индексов

```
DROP INDEX IF EXISTS idx_customerorders_customerid;

DROP INDEX IF EXISTS idx_deliveries_supplier_date;
```

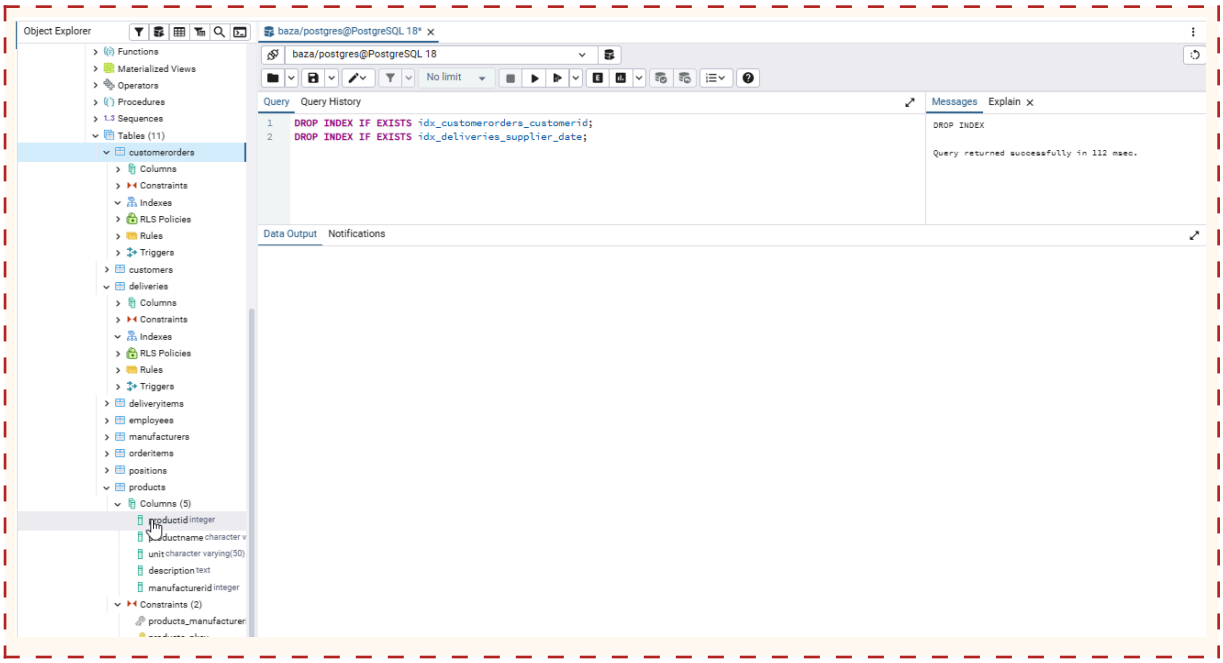

Рисунок 24 — Удаление индексов

9. Выводы

В ходе выполнения лабораторной работы были получены практические навыки работы с языком SQL в СУБД PostgreSQL и инструментом pgAdmin 4. Основные результаты:

— выполнено 6 SELECT-запросов разной сложности: с фильтрацией, соединением нескольких таблиц (включая JOIN на 5 таблиц), группировкой и агрегатными функциями, подзапросами и сортировкой;

— созданы 3 представления (v_employees_full, v_stock_status, v_orders_summary), упрощающих работу с типовыми отчётными запросами и инкапсулирующих логику соединений и вычисляемых полей (например, расчёт процента наценки);

— выполнены 3 модификации данных с подзапросами: INSERT для добавления отсутствующих строк в stock, UPDATE для пересчёта продажной цены и DELETE для удаления товаров без операций;

— изучены графическое представление плана запроса (вкладка Explain) и история выполненных запросов (вкладка Query History) в Query Tool pgAdmin 4;

— построены и проанализированы планы запросов с помощью EXPLAIN ANALYZE до и после создания простого индекса (по одному столбцу) и составного индекса (по двум столбцам); продемонстрировано переключение планировщика с Seq Scan на Index Scan при добавлении индексов;

— по итогам анализа индексы корректно удалены (DROP INDEX) для возврата БД в исходное состояние.

Освоены ключевые возможности подсистемы выполнения запросов PostgreSQL: GROUP BY и агрегаты, подзапросы, оператор JOIN различных видов, представления (VIEW), простые и составные индексы (CREATE INDEX), а также инструмент анализа планов EXPLAIN ANALYZE с графическим представлением.

10. Список источников

1. Документация PostgreSQL. Запросы [Электронный ресурс] // Официальный сайт PostgreSQL. URL: https://www.postgresql.org/docs/current/queries.html (дата обращения: 26.04.2026).

2. Документация PostgreSQL. CREATE VIEW [Электронный ресурс]. URL: https://www.postgresql.org/docs/current/sql-createview.html (дата обращения: 26.04.2026).

3. Документация PostgreSQL. CREATE INDEX [Электронный ресурс]. URL: https://www.postgresql.org/docs/current/sql-createindex.html (дата обращения: 26.04.2026).

4. Документация PostgreSQL. Using EXPLAIN [Электронный ресурс]. URL: https://www.postgresql.org/docs/current/using-explain.html (дата обращения: 26.04.2026).

5. Документация pgAdmin 4. Query Tool [Электронный ресурс]. URL: https://www.pgadmin.org/docs/pgadmin4/latest/query_tool.html (дата обращения: 26.04.2026).

6. Postgres Pro. Использование EXPLAIN [Электронный ресурс]. URL: https://postgrespro.ru/docs/postgresql/14/using-explain (дата обращения: 26.04.2026).